

Lecture Notes:

Autoencoders

CIC1205 - Machine Learning

Eduardo Bezerra

May 20, 2026

0 Contents

Before You Read	3
Learning Objectives	3
1 Introduction	4
1.1 From Supervised to Unsupervised Representation Learning	4
1.2 Motivation: Why Compress and Reconstruct?	4
2 Architecture	5
2.1 The Three Components	5
2.2 Overall Data Flow	6
2.3 Symmetry of Encoder and Decoder	6
2.4 Fully Connected vs. Convolutional Autoencoders	7
3 Training an Autoencoder	7
3.1 The Reconstruction Loss	7
3.2 Training Loop	8
4 The Vanilla Autoencoder	9
4.1 Architecture for MNIST	9
4.2 Implementation in PyTorch	9
4.3 Inspecting Reconstructions	11
5 Latent Space Analysis	12
5.1 What Does the Latent Space Represent?	12
5.2 Visualizing the Latent Space with t-SNE	12
5.3 Interpreting the Visualization	13
6 Denoising Autoencoders	14
6.1 Motivation	14
6.2 Why Denoising Helps	14
6.3 Noise Models	14
6.4 Implementation in PyTorch	15
6.5 Evaluating the Denoising Autoencoder	18
7 Implementation Notes and Best Practices	18
7.1 Initializing and Running the Model	18
7.2 Input Normalization	19
7.3 Choosing the Latent Dimension	19
8 Common Pitfalls and Debugging	20
8.1 Symptoms and Fixes	20
8.2 The Identity Solution	20
8.3 Spatial Size Mismatch in Convolutional Autoencoders	21
9 Connection to Deep Learning	21
9.1 The Autoencoder as a Foundation	21
9.2 Limitations of Standard Autoencoders	21

9.3 Variational Autoencoders	22
10 Final Remarks	23
10.1 Summary of Key Concepts	23
10.2 Recommended Next Steps	23

0 Before You Read

These notes build directly on the MLP lecture notes. The following topics should be familiar before proceeding:

- Multi-layer perceptrons: forward pass, activation functions, weight matrices.
- Backpropagation and gradient-based optimization (gradient descent, Adam).
- Loss functions: mean squared error and binary cross-entropy.
- Overfitting, regularization, and the train/validation/test split.
- Basic PyTorch: `nn.Module`, `nn.Sequential`, the training loop.

0 Learning Objectives

After studying these notes, you should be able to:

- explain the structure and purpose of an autoencoder in terms of encoder, latent space, and decoder;
- describe what is meant by unsupervised representation learning and why it is useful;
- state the reconstruction loss and explain why no labels are needed during training;
- explain the bottleneck constraint and its role in forcing the network to learn compact representations;
- describe the denoising autoencoder and explain how corrupting inputs during training improves robustness;
- interpret a latent space visualization and explain what clustering of classes reveals about learned representations;
- implement a vanilla autoencoder and a denoising autoencoder in PyTorch;
- identify common failure modes and apply appropriate fixes.

1 Introduction

1.1 From Supervised to Unsupervised Representation Learning

The MLPs studied so far are supervised models: they learn a mapping from inputs to labels by minimizing a loss that directly measures prediction error. This works well when labeled training data are available, but labeling data is expensive and time-consuming. In many practical settings, unlabeled data are plentiful while labeled examples are scarce.

A deeper issue is that the intermediate representations learned by a supervised MLP are shaped entirely by the labels. The network learns to retain only the information that is useful for a specific prediction task, discarding everything else. This limits the generality of the learned representations.

Unsupervised representation learning takes a different approach: instead of predicting labels, the network is trained to learn a compact and informative representation of the data itself, without any labels. The autoencoder is one of the simplest and most instructive architectures for this purpose.

Autoencoder

An **autoencoder** is a neural network trained to reproduce its input at its output, subject to an information bottleneck in the middle. It consists of two components: an **encoder** that compresses the input into a low-dimensional **latent representation**, and a **decoder** that reconstructs the input from that representation. Because the target output is the input itself, training requires no labels.

1.2 Motivation: Why Compress and Reconstruct?

At first glance, a network that simply copies its input seems pointless. The key is the bottleneck: the latent space has far fewer dimensions than the input, so the network cannot simply pass the input through unchanged. It must learn to retain only the most essential information.

This constraint forces the encoder to discover the underlying structure of the data: which features are redundant, which co-vary, and which carry the most information. The decoder then learns the inverse: how to recover a faithful reconstruction from the compressed representation.

The Photocopier Analogy

An autoencoder with a sufficiently large bottleneck could, in principle, act as a photocopier, simply memorizing and replaying any input. The bottleneck prevents this. Reducing the latent dimension forces the network to solve a compression problem, and in doing so it must discover structure in the data. The quality of the reconstruction is evidence that real patterns have been found.

The autoencoder framework is versatile. The same basic architecture supports a wide range of practical applications:

- **Dimensionality reduction**: the latent vector $\mathbf{z}$ serves as a compact feature rep-

resentation, analogous to what PCA computes linearly but potentially far more expressive.

- **Anomaly detection:** a model trained on normal data will produce high reconstruction error on anomalous inputs, since it has not learned to represent them.
- **Image denoising:** if the encoder is trained to ignore noise, the decoder produces a clean reconstruction from a corrupted input.
- **Data compression:** the encoder acts as a learned compressor and the decoder as a learned decompressor for a specific data distribution.

Entry Ticket 1

Think about the motivation for autoencoders before reading further.

1. PCA also produces a low-dimensional representation of data. What is the key difference between PCA and an autoencoder in terms of what kinds of structure each can capture?
2. If you train an autoencoder on images of handwritten digits and then pass in an image of a car, do you expect the reconstruction error to be high or low? Justify your answer.
3. Why is it meaningful that an autoencoder trained without any class labels can produce a latent space in which different digit classes form distinct clusters?

2 Architecture

2.1 The Three Components

An autoencoder has three conceptually distinct parts: the encoder, the latent space, and the decoder. Together they define a *compression–reconstruction* pipeline.

Encoder

The encoder is a neural network (typically an MLP or a convolutional network) that maps a high-dimensional input $\mathbf{x} \in \mathbb{R}^d$ to a low-dimensional latent vector $\mathbf{z} \in \mathbb{R}^k$, where $k \ll d$:

$$\mathbf{z} = f_{\phi}(\mathbf{x}),$$

where ϕ denotes the encoder parameters. The encoder applies a sequence of linear transformations and non-linear activations, progressively reducing dimensionality. Its output $\mathbf{z}$ is called the **latent representation**, **code**, or **embedding** of the input.

Latent Space (Bottleneck)

The latent space is the set of all possible values of $\mathbf{z}$. Its dimensionality k is a key hyperparameter: if k is too large, the network can memorize inputs without learning meaningful structure; if k is too small, the network cannot retain enough information to reconstruct the input faithfully.

The term *bottleneck* refers precisely to this constrained intermediate layer. It is the structural mechanism that prevents the autoencoder from learning the identity function and forces it to compress.

Decoder

The decoder is a neural network that maps the latent vector $\mathbf{z}$ back to the original input space, producing a reconstruction $\hat{\mathbf{x}} \in \mathbb{R}^d$:

$$\hat{\mathbf{x}} = g_{\psi}(\mathbf{z}),$$

where ψ denotes the decoder parameters. The decoder is typically a *mirrored* version of the encoder: if the encoder applies a sequence of downsampling operations, the decoder applies the corresponding upsampling operations in reverse order.

2.2 Overall Data Flow

Combining encoder and decoder, the full autoencoder computes:

$$\hat{\mathbf{x}} = g_{\psi}(f_{\phi}(\mathbf{x})).$$

The network is trained end-to-end so that $\hat{\mathbf{x}} \approx \mathbf{x}$. Both ϕ and ψ are learned simultaneously from unlabeled data. Figure 1 illustrates the architecture.

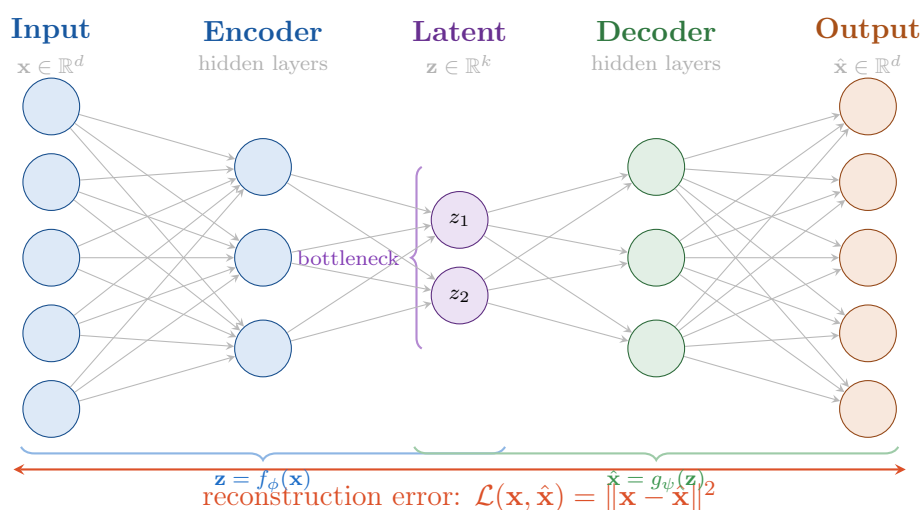


Figure 1: Autoencoder architecture. The encoder f_{ϕ} compresses the input $\mathbf{x}$ into a low-dimensional latent vector $\mathbf{z}$ (the bottleneck). The decoder g_{ψ} reconstructs $\hat{\mathbf{x}}$ from $\mathbf{z}$. The network is trained to minimize the reconstruction error between $\mathbf{x}$ and $\hat{\mathbf{x}}$.

2.3 Symmetry of Encoder and Decoder

A common design choice is to make the decoder a *mirror image* of the encoder: if the encoder applies layers with sizes $d \rightarrow h_1 \rightarrow h_2 \rightarrow k$, the decoder applies $k \rightarrow h_2 \rightarrow h_1 \rightarrow d$. This symmetry is not required by theory, but it provides a natural starting point and ensures that the decoder has the capacity to invert the encoder.

The Decoder is Not the Inverse of the Encoder

Although the decoder is designed to approximate the inverse of the encoder, it learns this inverse mapping from data. It does not compute an exact mathematical inverse.

In particular, because the bottleneck is lossy (information is discarded), the exact inverse does not exist for most inputs. The decoder learns the best approximate reconstruction it can achieve given the latent code.

2.4 Fully Connected vs. Convolutional Autoencoders

The architecture of the encoder and decoder can be adapted to the data type:

- **Fully connected (vanilla) autoencoder:** the input is flattened into a vector. Both encoder and decoder consist of fully connected (dense) layers. Suitable for tabular data and small images. For an MNIST image ($28 \times 28 = 784$ pixels), the encoder might reduce from 784 to 128 to 32 dimensions.
- **Convolutional autoencoder:** the encoder uses convolutional layers (which respect spatial structure) and the decoder uses *transposed convolutional layers* (class `ConvTranspose2d` in PyTorch) to upsample back to the original spatial dimensions. Preferred for image data because it exploits translation invariance and requires far fewer parameters.

Entry Ticket 2

Consider a fully connected autoencoder applied to MNIST images ($28 \times 28 = 784$ pixels).

1. The encoder has layers of sizes $784 \rightarrow 128 \rightarrow 32$. What are the dimensions of each weight matrix? How many learnable parameters does the encoder have in total (weights and biases)?
2. If the decoder mirrors the encoder with sizes $32 \rightarrow 128 \rightarrow 784$, how many parameters does the full autoencoder have?
3. If the latent dimension is reduced from 32 to 2, what do you expect to happen to the reconstruction quality? Would this always be worse? Is there ever a reason to use such a small latent space?

3 Training an Autoencoder

3.1 The Reconstruction Loss

Because the goal is to produce an output that matches the input, the natural training objective is to minimize the **reconstruction loss** between the input $\mathbf{x}$ and its reconstruction $\hat{\mathbf{x}}$:

$$\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \sum_{i=1}^d (x_i - \hat{x}_i)^2.$$

Over a dataset of m examples, the training objective is:

$$\min_{\phi, \psi} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\mathbf{x}^{(i)}, g_{\psi}(f_{\phi}(\mathbf{x}^{(i)}))).$$

Notice that **no labels appear anywhere in this objective**. The input $\mathbf{x}$ serves simultaneously as the model input and as the training target. This is why autoencoders are classified as unsupervised learning methods.

Choice of Reconstruction Loss

The appropriate reconstruction loss depends on the nature of the data:

- **Mean Squared Error (MSE)**: appropriate when the input values are continuous (e.g., real-valued sensor readings, normalized image pixels). The output layer of the decoder should use a linear activation (no constraint on output range).

$$\mathcal{L}_{\text{MSE}}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{d} \|\mathbf{x} - \hat{\mathbf{x}}\|^2.$$

- **Binary Cross-Entropy (BCE)**: appropriate when the input values are in $[0, 1]$ and can be interpreted as probabilities or normalized intensities (e.g., pixel values scaled to $[0, 1]$). The output layer should use a sigmoid activation.

$$\mathcal{L}_{\text{BCE}}(\mathbf{x}, \hat{\mathbf{x}}) = - \sum_{i=1}^d [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)].$$

Loss and Output Activation Must Match

The same rule applies here as in supervised MLPs: the output activation and the loss function must be compatible. For MSE, a linear (no) output activation is correct. For BCE, a sigmoid output activation is required. Using sigmoid with MSE is possible but creates a mismatched gradient scale. Using no activation with BCE is invalid because BCE requires outputs in $(0, 1)$.

3.2 Training Loop

The training procedure for an autoencoder is structurally identical to any MLP training loop, with one key difference: the labels used in the loss are the inputs themselves.

1. **Forward pass**: feed $\mathbf{x}$ through the encoder to get $\mathbf{z} = f_{\phi}(\mathbf{x})$, then through the decoder to get $\hat{\mathbf{x}} = g_{\psi}(\mathbf{z})$.
2. **Loss computation**: compute $\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}})$.
3. **Backward pass**: backpropagate the gradient through the decoder and then through the encoder, computing $\nabla_{\psi} \mathcal{L}$ and $\nabla_{\phi} \mathcal{L}$.
4. **Parameter update**: update both ϕ and ψ jointly using the optimizer.

Crucially, the encoder and decoder are trained together in a single optimization. Gradients flow from the reconstruction loss back through the decoder, through the bottleneck, and into the encoder. This end-to-end training ensures that the encoder learns to produce latent codes that the decoder can use effectively.

Common Misconception

It is tempting to think that the encoder should be trained first (to compress well), and then the decoder trained afterwards (to reconstruct from the compressed code). This two-stage approach would not work. The encoder needs to know what the decoder expects, and vice versa. End-to-end training allows both components to adapt jointly, converging on a shared representation that works for both compression and reconstruction.

4 The Vanilla Autoencoder

4.1 Architecture for MNIST

The simplest autoencoder uses only fully connected layers. For the MNIST handwritten digit dataset, each image is a 28×28 grayscale image, which when flattened gives an input vector of dimension $d = 784$.

A typical vanilla autoencoder for MNIST uses the following architecture:

Component	Layer	Output size	Activation
Encoder	Linear($784 \rightarrow 128$)	128	ReLU
	Linear($128 \rightarrow 32$)	32	None (linear)
Decoder	Linear($32 \rightarrow 128$)	128	ReLU
	Linear($128 \rightarrow 784$)	784	Sigmoid
	Unflatten to $(1, 28, 28)$	$(1, 28, 28)$	—

The encoder uses a ReLU activation in the hidden layer and a linear (no) activation at the bottleneck, so that the latent codes are unconstrained real vectors. The decoder's final layer uses a sigmoid activation to constrain the reconstructed pixel values to $(0, 1)$, compatible with normalized MNIST images.

4.2 Implementation in PyTorch

Listing 1 shows a complete PyTorch implementation of this vanilla autoencoder, including the training loop and a visualization function.

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torchvision import datasets, transforms
5 from torch.utils.data import DataLoader
6
7 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
8
9 # Dataset
10 transform = transforms.ToTensor()
```

```

11 train_dataset = datasets.MNIST(root='./data', train=True,
12                                transform=transform, download=True)
13 train_loader = DataLoader(train_dataset, batch_size=128, shuffle=
    True)
14
15 # Autoencoder architecture
16 class Autoencoder(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.encoder = nn.Sequential(
20             nn.Flatten(),
21             nn.Linear(28 * 28, 128),
22             nn.ReLU(),
23             nn.Linear(128, 32)          # bottleneck: 32-dim latent
                code
24         )
25         self.decoder = nn.Sequential(
26             nn.Linear(32, 128),
27             nn.ReLU(),
28             nn.Linear(128, 28 * 28),
29             nn.Sigmoid(),              # pixel values in (0, 1)
30             nn.Unflatten(1, (1, 28, 28))
31         )
32
33     def forward(self, x):
34         z = self.encoder(x)
35         return self.decoder(z)
36
37 model = Autoencoder().to(device)
38 criterion = nn.MSELoss()
39 optimizer = optim.Adam(model.parameters(), lr=1e-3)
40
41 # Training loop
42 num_epochs = 20
43 for epoch in range(num_epochs):
44     model.train()
45     for imgs, _ in train_loader:    # labels ignored: unsupervised
46         imgs = imgs.to(device)
47         recon = model(imgs)
48         loss = criterion(recon, imgs) # target = input itself
49
50         optimizer.zero_grad()
51         loss.backward()
52         optimizer.step()
53
54     print(f"Epoch [{epoch+1}/{num_epochs}] Loss: {loss.item():.4f}
    ")

```

Listing 1: Vanilla autoencoder for MNIST in PyTorch.

Labels Are Ignored

The loop iterates over `(imgs, _)`: the underscore discards the digit label. The loss compares `recon` against `imgs`, not against any external target. This single line encapsulates the entire notion of unsupervised training.

4.3 Inspecting Reconstructions

After training, the quality of learned representations can be assessed visually by comparing original images with their reconstructions. The function below extracts a batch of test images, passes them through the trained model, and displays the results side by side.

```

1 import matplotlib.pyplot as plt
2
3 def show_reconstructions(model, n=10):
4     model.eval()
5     test_imgs, _ = next(iter(train_loader))
6     test_imgs = test_imgs[:n].to(device)
7     with torch.no_grad():
8         recon = model(test_imgs)
9
10    fig, axes = plt.subplots(2, n, figsize=(n * 1.5, 3))
11    for i in range(n):
12        axes[0, i].imshow(test_imgs[i].cpu().squeeze(), cmap='gray',
13                           )
14        axes[0, i].axis('off')
15        axes[1, i].imshow(recon[i].cpu().squeeze(), cmap='gray')
16        axes[1, i].axis('off')
17    plt.suptitle("Top: Original | Bottom: Reconstruction")
18    plt.tight_layout()
19    plt.show()
20 show_reconstructions(model)

```

Listing 2: Visualization of reconstructed images.

A well-trained autoencoder produces reconstructions that are recognizable but slightly blurred, reflecting the information that was retained versus discarded by the bottleneck. This blurring is characteristic of MSE-trained autoencoders and is an active area of research in generative modeling.

Entry Ticket 3

After training the vanilla autoencoder described in Listing 1:

1. What would you expect to observe if the latent dimension is reduced from 32 to 2? Would a latent dimension of 2 have any practical advantage?
2. The reconstruction loss decreases during training, but the reconstructed images still look blurry. Is this a sign that training has failed? What property of the MSE loss contributes to this blurriness?
3. Describe what you would expect to happen if the bottleneck were removed

entirely (i.e., the latent dimension equals the input dimension of 784). Would the network still learn anything useful?

5 Latent Space Analysis

5.1 What Does the Latent Space Represent?

The latent space is the set of all codes $\mathbf{z}$ that the encoder produces for the training inputs. Since the encoder is a learned function, the geometry of the latent space reflects the structure of the data as discovered by the autoencoder. Points in the latent space that are close together correspond to inputs that the encoder has mapped to similar representations, and therefore to inputs that share the features the encoder found most important for reconstruction.

For MNIST, even though training is completely unsupervised (digit labels are never used), a well-trained autoencoder typically groups images of the same digit close together in the latent space. This happens because images of the same digit share visual structure, and the encoder learns to preserve that structure in the code.

5.2 Visualizing the Latent Space with t-SNE

For latent dimensions $k > 2$, direct visualization is impossible. A standard approach is to apply **t-SNE** (t-distributed Stochastic Neighbor Embedding), a nonlinear dimensionality reduction algorithm that projects high-dimensional data into 2D while approximately preserving local neighborhood structure.

The procedure is:

1. Run the trained encoder on all (or a large sample of) test images to obtain their latent codes $\{\mathbf{z}^{(i)}\}$.
2. Apply t-SNE to project these codes to 2D.
3. Plot the 2D projections, coloring each point by its true digit class.

Listing 3 shows this procedure.

```

1 from sklearn.manifold import TSNE
2 import matplotlib.pyplot as plt
3
4 def visualize_latent_space(model, data_loader, n_samples=2000):
5     model.eval()
6     latents, labels = [], []
7
8     with torch.no_grad():
9         for imgs, lbls in data_loader:
10             imgs = imgs.to(device)
11             z = model.encoder(imgs)          # obtain latent codes
12             latents.append(z.cpu())
13             labels.append(lbls)
14             if len(torch.cat(latents)) > n_samples:

```

```

15         break
16
17     latents = torch.cat(latents)[:n_samples]
18     labels = torch.cat(labels)[:n_samples]
19
20     # Project to 2D with t-SNE
21     tsne = TSNE(n_components=2, init='pca',
22                 learning_rate='auto', random_state=42)
23     coords = tsne.fit_transform(latents.numpy())
24
25     plt.figure(figsize=(8, 6))
26     scatter = plt.scatter(coords[:, 0], coords[:, 1],
27                           c=labels, cmap='tab10', s=10, alpha=0.7)
28     plt.legend(*scatter.legend_elements(), title="Digits")
29     plt.title("t-SNE projection of autoencoder latent space")
30     plt.tight_layout()
31     plt.show()
32
33 visualize_latent_space(model, train_loader)

```

Listing 3: t-SNE visualization of the latent space.

5.3 Interpreting the Visualization

A t-SNE plot of the autoencoder latent space typically reveals well-separated clusters, each corresponding to one digit class. This result is informative in two ways.

First, it confirms that the autoencoder has learned **semantically meaningful features**. Although digit identity was never part of the training signal, the network has discovered representations that correlate strongly with digit class, because digits of the same class look similar and therefore compress similarly.

Second, it illustrates that **the geometry of the latent space encodes similarity**. Digits that look visually similar (e.g., 4 and 9, or 3 and 8) tend to have nearby latent codes, while visually dissimilar digits are farther apart.

Unsupervised Feature Discovery

A t-SNE visualization showing class-discriminative clustering in the latent space, despite training being entirely unsupervised, is evidence that the autoencoder has discovered the intrinsic structure of the data. This structure arises from visual similarity, not from labels. It suggests that the learned representations could serve as useful features for downstream tasks such as classification, clustering, or retrieval.

Entry Ticket 4

Reflect on the t-SNE latent space visualization.

1. The autoencoder was trained with no label information, yet the t-SNE plot shows clusters corresponding to digit classes. What does this tell you about the relationship between reconstruction quality and class structure in the

data?

2. Suppose you use the latent codes $\mathbf{z}$ as features for a k -nearest-neighbor classifier. Would you expect this to work well? What is the risk compared to using the original pixel values?
3. t-SNE is a *visualization* tool, not a distance metric. Why is it important not to over-interpret the distances between clusters in a t-SNE plot?

6 Denoising Autoencoders

6.1 Motivation

A plain autoencoder trained to minimize reconstruction loss may learn a trivial solution if the bottleneck is large: simply copying the input through the network with minimal compression. Even with a tight bottleneck, the latent representations may not be particularly robust: small perturbations to the input can produce large changes in the code.

The **denoising autoencoder** addresses both issues by corrupting the input before passing it to the encoder, while still using the clean original as the reconstruction target. The network must therefore learn to ignore noise and recover the underlying clean signal.

Denoising Autoencoder

A **denoising autoencoder** is trained by:

1. Corrupting each training input $\mathbf{x}$ to obtain a noisy version $\tilde{\mathbf{x}}$;
2. Passing $\tilde{\mathbf{x}}$ through the encoder-decoder pipeline to obtain $\hat{\mathbf{x}}$;
3. Minimizing the reconstruction loss between the clean input $\mathbf{x}$ and the reconstruction $\hat{\mathbf{x}}$.

The loss function is $\mathcal{L}(\mathbf{x}, g_{\psi}(f_{\phi}(\tilde{\mathbf{x}})))$. The network never sees the clean input directly during training; it must learn to recover it from the corrupted version.

6.2 Why Denoising Helps

The denoising objective acts as a regularizer. By forcing the network to reconstruct clean inputs from corrupted ones, it encourages the encoder to:

- Learn representations that capture stable, noise-resistant features of the data.
- Ignore irrelevant variations (noise) that do not help reconstruction.
- Build a latent space that is more robust and more disentangled than a plain autoencoder.

The training signal is richer: the network receives explicit feedback on which features of the input are essential (those needed for reconstruction) versus which can be discarded (noise).

6.3 Noise Models

Different types of corruption can be used:

- **Additive Gaussian noise:** $\tilde{\mathbf{x}} = \text{clip}(\mathbf{x} + \epsilon, 0, 1)$ where $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$.
- **Salt-and-pepper noise:** randomly set a fraction of pixel values to 0 or 1.
- **Masking noise:** randomly set a fraction of input features to 0.

The choice of noise type should match the type of corruption expected in the application. For image denoising, Gaussian noise is the most common. The noise level σ (or the fraction of corrupted inputs) is a hyperparameter: too little noise provides insufficient regularization; too much noise may make reconstruction impossible.

Figure 2 illustrates the denoising training procedure.

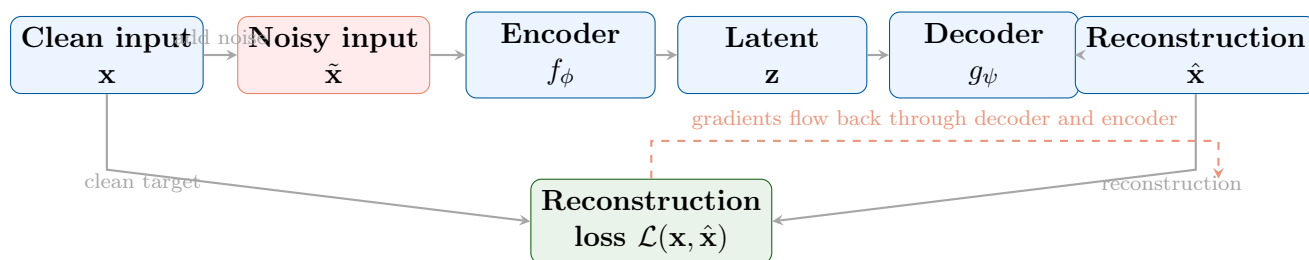


Figure 2: Training procedure for a denoising autoencoder. The clean input $\mathbf{x}$ is corrupted to produce $\tilde{\mathbf{x}}$, which is fed to the encoder–decoder pipeline. The reconstruction loss compares the output $\hat{\mathbf{x}}$ against the clean original $\mathbf{x}$, not against the corrupted version. Gradients flow back through both the decoder and the encoder.

6.4 Implementation in PyTorch

The denoising autoencoder notebook uses a **convolutional** architecture for the encoder and decoder, which is more appropriate for image data. The encoder applies three convolutional layers followed by two fully connected layers; the decoder mirrors this with two fully connected layers followed by three transposed convolutional layers.

Noise Function

```

1 def add_noise(inputs, noise_factor=0.3):
2     # torch.randn_like creates noise of the same shape as inputs
3     noise = inputs + torch.randn_like(inputs) * noise_factor
4     # Clip to [0, 1] to keep pixel values valid
5     noise = torch.clip(noise, 0., 1.)
6     return noise

```

Listing 4: Additive Gaussian noise for the denoising autoencoder.

Convolutional Encoder

```

1 class Encoder(nn.Module):
2     def __init__(self, encoded_space_dim):
3         super().__init__()
4         # Convolutional layers (downsampling)
5         self.encoder_cnn = nn.Sequential(

```



```

6         nn.Conv2d(1, 8, 3, stride=2, padding=1),
7         nn.ReLU(True),
8         nn.Conv2d(8, 16, 3, stride=2, padding=1),
9         nn.BatchNorm2d(16),
10        nn.ReLU(True),
11        nn.Conv2d(16, 32, 3, stride=2, padding=0),
12        nn.ReLU(True)
13    )
14    # Flatten and fully connected layers
15    self.flatten = nn.Flatten(start_dim=1)
16    self.encoder_lin = nn.Sequential(
17        nn.Linear(3 * 3 * 32, 128),
18        nn.ReLU(True),
19        nn.Linear(128, encoded_space_dim) # bottleneck
20    )
21
22    def forward(self, x):
23        x = self.encoder_cnn(x)
24        x = self.flatten(x)
25        return self.encoder_lin(x)

```

Listing 5: Convolutional encoder for the denoising autoencoder.

Convolutional Decoder

```

1 class Decoder(nn.Module):
2     def __init__(self, encoded_space_dim):
3         super().__init__()
4         # Fully connected layers
5         self.decoder_lin = nn.Sequential(
6             nn.Linear(encoded_space_dim, 128),
7             nn.ReLU(True),
8             nn.Linear(128, 3 * 3 * 32),
9             nn.ReLU(True)
10        )
11        self.unflatten = nn.Unflatten(dim=1, unflattened_size=(32,
12            3, 3))
13        # Transposed convolutional layers (upsampling)
14        self.decoder_conv = nn.Sequential(
15            nn.ConvTranspose2d(32, 16, 3, stride=2, output_padding
16                =0),
17            nn.BatchNorm2d(16),
18            nn.ReLU(True),
19            nn.ConvTranspose2d(16, 8, 3, stride=2,
20                padding=1, output_padding=1),
21            nn.BatchNorm2d(8),
22            nn.ReLU(True),
23            nn.ConvTranspose2d(8, 1, 3, stride=2,
24                padding=1, output_padding=1)
25        )

```

```

25     def forward(self, x):
26         x = self.decoder_lin(x)
27         x = self.unflatten(x)
28         x = self.decoder_conv(x)
29         return torch.sigmoid(x) # output pixel values in (0, 1)

```

Listing 6: Convolutional decoder using transposed convolutions.

Training Loop

The key difference from the vanilla autoencoder training loop is that the encoder receives the *noisy* image, while the loss is computed against the *clean* image.

```

1  def train_epoch_den(encoder, decoder, device, dataloader,
2                          loss_fn, optimizer, noise_factor=0.3):
3      encoder.train()
4      decoder.train()
5      train_losses = []
6
7      for image_batch, _ in dataloader: # labels ignored
8          # Corrupt the input
9          image_noisy = add_noise(image_batch, noise_factor).to(
10                                  device)
11
12         # Forward pass on noisy input
13         z = encoder(image_noisy)
14         decoded = decoder(z)
15
16         # Loss against the CLEAN original
17         loss = loss_fn(decoded, image_noisy)
18
19         optimizer.zero_grad()
20         loss.backward()
21         optimizer.step()
22         train_losses.append(loss.detach().cpu().item())
23
24     return np.mean(train_losses)

```

Listing 7: Training loop for the denoising autoencoder.

Transposed Convolution

A **transposed convolution** (ConvTranspose2d) is the decoder's counterpart to the encoder's regular convolution. While a regular convolution with stride > 1 reduces spatial dimensions (downsampling), a transposed convolution increases spatial dimensions (upsampling). It learns an upsampling filter from data, rather than using a fixed interpolation. This allows the decoder to reconstruct spatial structure in the output image.

Entry Ticket 5

Compare the vanilla autoencoder (Section 4) with the denoising autoencoder.

1. In the training function `train_epoch_den` (Listing 7), the loss is computed as `loss_fn(decoded, image_noisy)`. Is this the correct target for the denoising objective? If not, what should the second argument be, and why?
2. The denoising autoencoder uses `BatchNorm2d` in both encoder and decoder. What problem does batch normalization address, and why might it be especially helpful in deep convolutional networks?
3. After 30 training epochs, reconstructed images still show some imperfections. Is this expected behavior, or does it indicate a problem? What architectural or training changes might improve reconstruction quality?

6.5 Evaluating the Denoising Autoencoder

Evaluation compares three images side by side: the clean original, the corrupted input, and the reconstruction. If the autoencoder has learned well, the reconstruction should be visually closer to the clean original than to the corrupted input.

The model can also be evaluated quantitatively using the test set loss. Since the autoencoder is unsupervised, there is no classification accuracy to report; the primary metric is the reconstruction loss on held-out data.

Generating New Images from Random Codes

An additional way to probe what the autoencoder has learned is to generate new images from random latent codes. Since the decoder has learned to map codes to images, sampling $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2 \mathbf{I})$ (where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are estimated from the encoded training data) and passing these samples through the decoder produces synthetic images.

However, a standard (non-variational) autoencoder does not enforce any structure on the latent space. The latent codes learned by a plain autoencoder can be scattered irregularly, and random samples may land in regions of the latent space that the decoder has never seen during training, producing nonsensical outputs. This limitation motivates the Variational Autoencoder (VAE), discussed in Section 9.

7 Implementation Notes and Best Practices

7.1 Initializing and Running the Model

When using separate `Encoder` and `Decoder` modules (rather than a single `Autoencoder` module), the optimizer must include the parameters of both:

```

1 d = 4 # latent space dimension
2
3 encoder = Encoder(encoded_space_dim=d)
4 decoder = Decoder(encoded_space_dim=d)
5
6 loss_fn = nn.MSELoss()
7
```

```

8 params_to_optimize = [
9     {'params': encoder.parameters()},
10    {'params': decoder.parameters()}
11 ]
12 optimizer = torch.optim.Adam(params_to_optimize, lr=1e-3)
13
14 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
15 encoder.to(device)
16 decoder.to(device)

```

Listing 8: Setting up the optimizer for separate encoder and decoder modules.

7.2 Input Normalization

As with any neural network, inputs should be normalized. For image data in $[0, 1]$ (e.g., using `transforms.ToTensor()`, which divides pixel values by 255), normalization is already implicitly applied. For tabular data or other continuous inputs, standard normalization (zero mean, unit variance) should be applied before training.

7.3 Choosing the Latent Dimension

The latent dimension k is one of the most important hyperparameters. There is a fundamental trade-off:

- **Larger k :** lower reconstruction loss, but latent codes may be less compact and less structured. The risk of learning a trivial near-identity mapping increases.
- **Smaller k :** higher compression, potentially more structured latent space, but higher reconstruction loss. At some point, k is too small to support faithful reconstruction.

A practical approach is to plot reconstruction loss as a function of k and select the smallest k that achieves acceptable reconstruction quality. For MNIST, values in the range $k = 2$ to $k = 64$ are commonly used, depending on the application.

Practical Guidelines for Autoencoders

- **Architecture:** start with a symmetric encoder-decoder pair; use fully connected layers for tabular data, convolutional layers for images.
- **Bottleneck:** choose k based on the task; use a validation reconstruction loss to guide selection.
- **Activation:** ReLU for hidden layers; sigmoid for the output layer when inputs are in $[0, 1]$; linear otherwise.
- **Loss:** MSE for continuous inputs; BCE for binary or normalized $[0, 1]$ inputs.
- **Optimizer:** Adam with learning rate 10^{-3} is a reliable default.
- **Normalization:** always normalize inputs; use `BatchNorm` in convolutional encoders/decoders to stabilize training.
- **Denoising:** add noise during training if robustness or regularization is needed; use a noise level that is challenging but not overwhelming.
- **Evaluation:** inspect reconstructions visually; use t-SNE for latent space anal-

ysis.

8 Common Pitfalls and Debugging

8.1 Symptoms and Fixes

Table 1 summarizes common problems encountered when training autoencoders and their likely causes.

Table 1: Common symptoms, likely causes, and fixes when training autoencoders.

Symptom	Likely cause	Fix
Loss does not decrease	LR too small; wrong loss/activation pairing	Increase LR; check output activation
Reconstructions are all gray/blurry	Bottleneck too small	Increase latent dimension
Reconstructions are nearly perfect	Bottleneck too large; no compression	Reduce latent dimension
Loss diverges (NaN)	LR too large; missing normalization	Reduce LR; add normalization
Random samples from latent space look like noise	Irregular latent space (not a DAE or VAE)	Use VAE; or estimate latent statistics from training data
Denoising: reconstruction looks noisy	Noise factor too high	Reduce noise factor
Denoising: reconstruction unchanged from input	Noise factor too low	Increase noise factor
Convolutional decoder produces wrong spatial size	Incorrect padding/stride settings	Verify <code>ConvTranspose2d</code> output size formula

8.2 The Identity Solution

A critical failure mode unique to autoencoders is the **identity solution**: if the bottleneck dimension equals or exceeds the input dimension, the network can learn to copy the input without any compression. The loss reaches near-zero but nothing useful has been learned. The fix is to always ensure $k \ll d$.

Even with $k < d$, if the encoder architecture is too powerful relative to the bottleneck constraint, the network may find near-lossless codes that do not generalize. In such cases, regularization (such as the denoising approach or the variational approach described in Section 9) is needed.

Common Misconception

A very low reconstruction loss does not necessarily mean the autoencoder has learned good representations. It may simply mean the bottleneck is too large. Always evaluate the learned representations directly, either through visualization (t-SNE), downstream task performance, or qualitative inspection of reconstructions.

8.3 Spatial Size Mismatch in Convolutional Autoencoders

When using transposed convolutions, the output spatial size depends on the input size, kernel size, stride, padding, and output padding parameters. Getting these wrong leads to size mismatches between encoder outputs and decoder inputs. The general formula for the output size of a `ConvTranspose2d` operation is:

$$H_{\text{out}} = (H_{\text{in}} - 1) \cdot \text{stride} - 2 \cdot \text{padding} + \text{kernel_size} + \text{output_padding}.$$

Always verify that the decoder produces tensors of the correct spatial dimensions by printing intermediate tensor shapes during development.

9 Connection to Deep Learning

9.1 The Autoencoder as a Foundation

The autoencoder establishes a set of concepts that underpin many modern deep learning architectures:

- **Representation learning:** the idea that a network can learn useful features from data without explicit supervision extends to self-supervised and contrastive learning methods.
- **Encoder–decoder structure:** the pattern of encoding to a compact representation and decoding back appears in sequence-to-sequence models, image segmentation networks (U-Net), and neural machine translation.
- **Bottleneck as regularization:** constraining the information flow through a network to prevent memorization is also the basis of the information bottleneck principle in deep learning theory.

9.2 Limitations of Standard Autoencoders

Standard autoencoders have two important limitations that motivate further development:

Irregular Latent Space

Because no constraint is placed on the latent space during training, the codes $\mathbf{z}$ can be distributed in any arbitrary way. The latent space may have large empty regions, and random samples from these regions will be decoded into nonsensical outputs. This makes standard autoencoders unsuitable for *generative* tasks (producing new data by sampling).

Lack of Interpretability

Individual dimensions of the latent vector do not correspond to interpretable factors of variation. The latent codes are useful for reconstruction and for tasks that use them as features, but they do not provide a transparent decomposition of the data.

9.3 Variational Autoencoders

The **Variational Autoencoder (VAE)** addresses both limitations by modifying the training objective:

- Instead of encoding each input to a point $\mathbf{z}$, the encoder outputs the parameters $(\boldsymbol{\mu}, \boldsymbol{\sigma})$ of a Gaussian distribution, and $\mathbf{z}$ is sampled from this distribution.
- A **KL-divergence regularization term** is added to the loss, penalizing the posterior distribution of each latent code for deviating from a standard normal prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

The combined loss is:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}})}_{\text{reconstruction}} + \underbrace{D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\text{regularization}}.$$

The KL term enforces that latent codes are distributed approximately according to $\mathcal{N}(\mathbf{0}, \mathbf{I})$, making the latent space smooth and continuous. This enables meaningful generation: new samples can be obtained by sampling $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and decoding them.

From Autoencoders to VAEs

The VAE is not merely an improved autoencoder: it is a probabilistic generative model with a sound theoretical foundation in variational inference. Understanding the standard autoencoder is, however, the essential prerequisite for VAEs. The architectural structure (encoder, bottleneck, decoder, reconstruction loss) is identical; the key innovation is the probabilistic interpretation of the latent space and the KL regularization term.

Table 2: Comparison of autoencoder variants discussed in these notes and their primary use cases.

Model	Key idea	Primary use case
Vanilla autoencoder	Minimize reconstruction loss through a bottleneck	Dimensionality reduction, feature learning
Denoising autoencoder	Train on corrupted inputs, reconstruct clean originals	Robust representations, image denoising
Variational autoencoder (VAE)	Stochastic latent space + KL regularization	Generative modeling, smooth latent space

Entry Ticket 6

Reflect on the connection between autoencoders and other models you have studied.

1. The denoising autoencoder is described as learning more robust representations than a plain autoencoder. Can you relate this to any regularization technique from the MLP notes (e.g., dropout, weight decay)? What is similar and what is different?
2. Why is the standard autoencoder *not* a generative model, even though it can produce reconstructions of training inputs? What property would a true generative model need to have?
3. An encoder trained as part of an autoencoder produces embeddings that can be used as features for a downstream classifier. What is the advantage of this approach over training the classifier directly on raw input features, particularly when labeled data are scarce?

10 Final Remarks

10.1 Summary of Key Concepts

- An **autoencoder** is a neural network trained to reconstruct its input through a low-dimensional bottleneck. Training requires no labels: the input serves as both input and target.
- The **encoder** compresses input $\mathbf{x}$ to a latent code $\mathbf{z}$; the **decoder** reconstructs $\hat{\mathbf{x}}$ from $\mathbf{z}$. Both are learned end-to-end.
- The **bottleneck constraint** forces the network to discover compact, meaningful representations by preventing simple copying of the input.
- The **reconstruction loss** (MSE or BCE, matched to the output activation) is the sole training objective. No class labels are needed.
- The **denoising autoencoder** improves robustness by training on corrupted inputs while targeting clean originals, acting as an effective regularizer.
- **Latent space visualization** (e.g., via t-SNE) reveals that autoencoders learn semantically meaningful representations even without supervision: similar inputs cluster together.
- Standard autoencoders have an irregular latent space, making generation from random codes unreliable. The **Variational Autoencoder (VAE)** addresses this by regularizing the latent space with a KL-divergence term.
- The encoder–decoder structure is a foundational pattern in deep learning, appearing in image segmentation, sequence-to-sequence translation, and beyond.

10.2 Recommended Next Steps

These notes prepare the student for the following topics:

Autoencoders → Variational Autoencoders (VAEs) → Generative Adversarial Networks (GANs) → Diffusion Models

Key Takeaways

- Autoencoders perform unsupervised representation learning: meaningful structure is discovered from data without labels.
- The bottleneck is the mechanism that makes this learning non-trivial and forces compression.
- Denoising adds a simple but powerful regularization signal that improves representation quality.
- The learned latent representations can serve downstream tasks (classification, clustering, retrieval) even when labeled data are scarce.
- All autoencoder variants are trained by backpropagation through the full encoder–decoder pipeline, using the same tools (Adam, gradient descent) as supervised MLPs.